

Arquitetura do Sistema

1. Visão Geral

O sistema Fruits é uma aplicação web para uma loja de hortifrúti local. Sua arquitetura pode ser classificada como uma Single Page Application (SPA) com backend monolítico que expõe uma API REST.

- Frontend (SPA): A interface do usuário é carregada uma única vez no navegador. Todo o código HTML, CSS e JavaScript é entregue de uma só vez, e a partir daí a aplicação manipula o DOM dinamicamente e se comunica com o servidor de forma assíncrona para buscar dados. Isso é evidenciado pelo uso de fetch no script.js e pela ausência de recarregamento completo da página durante a navegação ou ao adicionar itens ao carrinho.
- Backend (Monolito): O servidor, implementado em Node.js com o framework Express (server.js), é um monolito: em tempo de execução, todo o sistema é um único processo. Ele é responsável por servir os arquivos estáticos do frontend e por fornecer os endpoints da API (/api/produtos, /api/produtos/:id, /api/categorias).

2. Padrão Arquitetural: MVC Adaptado para a Web

- Modelo: É representado pela API REST e pelos dados dos produtos armazenados no servidor. O array de produtos em server.js contém os dados de domínio (nome, preço, imagem, categoria) e as rotas Express implementam a lógica de acesso a esses dados. O frontend não acessa o "banco de dados" diretamente; ele consome o modelo por meio de requisições HTTP.
- Visão: Corresponde ao HTML e CSS (index.html e style.css). A estrutura da página (cabeçalho, seções, modal do carrinho, footer) é definida em HTML, enquanto a estilização e o layout responsivo ficam a cargo do CSS. A Visão é atualizada dinamicamente pelo JavaScript.
- Controlador: É implementado pelo JavaScript do frontend (script.js). Ele é responsável por capturar eventos do usuário (cliques em "Adicionar ao Carrinho", abertura do modal, alteração de quantidade) e, a partir deles, ou buscar dados na API (Modelo) ou atualizar a interface (Visão). Por exemplo, a função adicionarAoCarrinho(id) atua como um controlador: ela reage ao evento de clique, consulta a API para obter os dados do produto e, em seguida, atualiza o estado do carrinho (modelo local) e re-renderiza o modal (visão).

3. Arquitetura em Camadas

- Camada de Apresentação (Frontend): Responsável pela interface com o usuário. Inclui a estrutura HTML, os estilos CSS e a lógica de interação imediata (DOM). É a camada que o usuário vê e com a qual interage diretamente.

- Camada de Lógica de Aplicação (Frontend): Responsável pelo gerenciamento do estado do carrinho, cálculos de total, validações simples e coordenação das chamadas à API. Essa camada está concentrada no `script.js`, que mantém o array carrinho e implementa as funções de adicionar, remover, limpar e calcular total.
- Camada de Dados / Serviços (Backend): Responsável por fornecer os dados dos produtos por meio de uma API REST. O `server.js` expõe os endpoints e contém a "base de dados" (um array em memória) que simula a persistência.

4. Justificativa das Decisões Arquiteturais

- Monolito é adequado para começar: Um monolito oferece a rota mais rápida para um produto funcional, com menos complexidade operacional.
- SPA para melhor experiência do usuário: A escolha de uma SPA se justifica pela necessidade de uma interface mais responsiva. Em aplicações web tradicionais, cada ação do usuário gera uma requisição ao servidor e uma recarga completa da página, o que torna a experiência menos fluida. No Fruits, ações como adicionar um item ao carrinho ou abrir o modal não recarregam a página, proporcionando uma experiência mais próxima de um aplicativo nativo.
- MVC adaptado para web: A adoção de uma estrutura inspirada em MVC para organizar o código do frontend segue uma tendência histórica. A partir dos anos 2000, frameworks web populares passaram a usar a terminologia MVC para nomear seus componentes, mesmo que a arquitetura subjacente se assemelhasse a sistemas de três camadas. No Fruits, essa organização facilita a compreensão e a manutenção do código.

5. Módulos e Tecnologias

Módulo	Arquivo	Responsabilidade	Tecnologia
Interface de Usuário	<code>index.html</code> , <code>style.css</code>	Estrutura semântica, layout responsivo, estilização de cards, modal e formulários.	HTML5, CSS3 (Flexbox, variáveis CSS, media queries)
Controlador e Lógica de Frontend	<code>script.js</code>	Gerenciamento do estado do carrinho, consumo da API, renderização dinâmica, tratamento de eventos.	JavaScript (ES6+), Fetch API, DOM API

API REST	server.js	Exposição dos endpoints de produtos e categorias, simulação de persistência.	Node.js, Express 5.2.1, CORS
Dados	Array em server.js	Armazenamento dos produtos com id, nome, preço, imagem e categoria.	Estrutura de dados em memória
Ícones e Imagens	CDN e Unsplash	Elementos visuais da interface.	Font Awesome 6.7.2, imagens externas

6. Limitações e Evoluções Futuras

- Ausência de persistência real: Os dados residem em memória e são reiniciados a cada execução do servidor. Uma evolução natural seria a integração com um banco de dados (PostgreSQL, MySQL ou MongoDB).
- Monolito com acoplamento de deploy: Frontend e backend são implantados juntos. Se o sistema crescer a ponto de times distintos precisarem de autonomia de deployment, a migração para microsserviços poderia ser considerada, seguindo o padrão Strangler Fig sugerido no capítulo.
- Segurança e autenticação: Não há autenticação de usuários nem validação de dados no backend. Em um cenário de produção, seria necessário implementar autenticação e validações de entrada.
- Escalabilidade: O monolito atual não é escalável horizontalmente sem ajustes. Microsserviços ou, pelo menos, a separação da API em um serviço independente permitiriam escalar apenas as partes com maior demanda.

7. Conclusão

A arquitetura do sistema Fruits é uma aplicação web moderna baseada em SPA com backend monolítico, organizada segundo uma adaptação do padrão MVC para a web e disposta logicamente em três camadas (apresentação, lógica de aplicação e dados). Essa estrutura equilibra simplicidade, clareza e funcionalidade, sendo adequada para o porte do projeto e para os objetivos da disciplina. As decisões arquiteturais foram fundamentadas nos conceitos apresentados no Capítulo 7 de Engenharia de Software Moderna, especialmente no que se refere à escolha de um monolito para projetos em estágio inicial e ao uso de SPA para melhorar a experiência do usuário. O sistema está preparado para evoluções futuras, como a migração para microsserviços ou a introdução de persistência real, caso a demanda cresça.